

Hybrid Search Fusion, Worked Out by Hand

Lexical (BM25) and semantic (cosine) rankings disagree and live on different scales. Two fusion methods — weighted sum and reciprocal-rank fusion — combine them, and they pick different winners.

What this document does. A strong RAG server runs both BM25 and vector search, then *fuses* the two result lists. This document takes three documents whose lexical and semantic rankings disagree and works out both standard fusions by hand — min-max weighted sum and Reciprocal Rank Fusion (RRF) — and shows, with the numbers, why they crown different documents.

Concepts covered: score-scale mismatch · min-max normalisation · weighted-sum fusion · Reciprocal Rank Fusion · peak-rank vs consistency · choosing α and k .

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: two rankers that disagree

Three candidate documents are scored by both engines (BM25 from the lexical document, cosine from the semantic one):

doc	BM25	BM25 rank	cosine	cosine rank
<i>A</i>	1.94	1	0.20	3
<i>B</i>	0.50	3	0.99	1
<i>C</i>	1.20	2	0.95	2

A wins on keywords but is semantically weak; *B* is the reverse; *C* is second in *both*. There is no single obvious winner — fusion has to decide.

Intuition

Lexical and semantic search fail differently. BM25 nails exact terminology but is blind to paraphrase; cosine captures meaning but can drift on rare exact tokens (a product code, an error number). Fusing them covers each one's blind spot. The only question is *how* to combine a BM25 score of 1.94 with a cosine of 0.99 — numbers that don't live on the same scale.

2 Step 1 — Why you can't just add the scores

BM25 is unbounded ($[0, \infty)$); cosine is bounded ($[-1, 1]$). Adding raw $1.94 + 0.20 = 2.14$ for *A* versus $0.50 + 0.99 = 1.49$ for *B* would let BM25's larger numbers dominate purely because of *units*, not relevance. The two scores must be put on a common scale first — or compared by *rank* instead of value.

3 Step 2 — Fusion I: min-max weighted sum

Rescale each engine's scores to $[0, 1]$ with min-max normalisation,

$$\tilde{s} = \frac{s - s_{\min}}{s_{\max} - s_{\min}},$$

then blend with weight α on lexical, $1 - \alpha$ on semantic.

Normalise BM25 (min = 0.50, max = 1.94, range 1.44):

$$\tilde{A} = \frac{1.94-0.50}{1.44} = 1.000, \quad \tilde{C} = \frac{1.20-0.50}{1.44} = 0.486, \quad \tilde{B} = \frac{0.50-0.50}{1.44} = 0.000.$$

Normalise cosine (min = 0.20, max = 0.99, range 0.79):

$$\tilde{B} = \frac{0.99-0.20}{0.79} = 1.000, \quad \tilde{C} = \frac{0.95-0.20}{0.79} = 0.949, \quad \tilde{A} = \frac{0.20-0.20}{0.79} = 0.000.$$

Blend at $\alpha = 0.5$, score = $\alpha\tilde{s}_{\text{bm25}} + (1 - \alpha)\tilde{s}_{\text{cos}}$:

$$A = 0.5(1.0) + 0.5(0.0) = 0.500, \quad B = 0.5(0.0) + 0.5(1.0) = 0.500, \quad C = 0.5(0.486) + 0.5(0.949) = \mathbf{0.718}.$$

Weighted-sum winner: C , with A and B tied behind it.

Intuition

Weighted sum rewards *consistency*. C was never first, but it was strong in *both* engines, and averaging two strong-ish normalised scores beats averaging one perfect and one zero. If you trust that a truly relevant document should look good to both rankers, weighted sum is the right instinct.

4 Step 3 — Fusion II: Reciprocal Rank Fusion

RRF ignores scores entirely and uses only *ranks*, which sidesteps the scale problem. With a damping constant $k = 60$ (the standard),

$$\text{RRF}(d) = \sum_{\text{engines}} \frac{1}{k + \text{rank}_{\text{engine}}(d)}.$$

$$A : \frac{1}{60+1} + \frac{1}{60+3} = 0.016393 + 0.015873 = 0.032266,$$

$$B : \frac{1}{60+3} + \frac{1}{60+1} = 0.015873 + 0.016393 = 0.032266,$$

$$C : \frac{1}{60+2} + \frac{1}{60+2} = 0.016129 + 0.016129 = 0.032258.$$

RRF winner: A and B (tied), with C *last* by a hair.

doc	weighted sum	RRF	disagree?
A	0.500	0.032266	top in RRF
B	0.500	0.032266	top in RRF
C	0.718	0.032258	top in weighted sum, <i>last</i> in RRF

Watch out

The two methods reverse the verdict on C . Weighted sum makes C the winner; RRF makes it the loser. This is not a bug — it is the methods' values showing through. The choice of

fusion is a modelling decision, not a formality, and you should pick it deliberately.

5 Step 4 — Peak-rank vs consistency

Why does RRF punish C ? Because $\frac{1}{k+\text{rank}}$ is *convex* in rank: the jump from rank 1 to 2 is worth more than from 2 to 3. Being #1 in one engine (A, B) earns a bigger reciprocal than being #2 in both (C). RRF rewards a *peak*; weighted sum rewards an *average*.

Mini-example — this operation in isolation

The reciprocals at $k = 60$: rank 1 $\rightarrow 0.016393$, rank 2 $\rightarrow 0.016129$, rank 3 $\rightarrow 0.015873$. The gain for moving 2 $\rightarrow$ 1 is 0.000264; for 3 $\rightarrow$ 2 it is 0.000256. Small here because $k = 60$ is large and flattens the curve — lower k (say $k = 1$: 0.5, 0.333, 0.25) sharpens it, making top ranks count far more. k is the dial between “only the top matters” and “deep ranks still count.”

Intuition

Use **RRF** when the two engines’ scores are on wildly different or unstable scales (the common case) — it is robust because it throws scores away and keeps order. Use **weighted sum** when scores are calibrated and you want a tunable α to favour lexical or semantic. Many production stacks default to RRF for exactly the robustness reason, then add a reranker on the fused top- k .

6 The argument, at a glance

#	Idea	Formula	Winner
1	Scales mismatch	BM25 $[0, \infty)$ vs $\cos [-1, 1]$	—
2	Min-max normalise	$(s - s_{\min}) / (s_{\max} - s_{\min})$	—
2	Weighted sum	$\alpha \tilde{s}_{\text{bm25}} + (1 - \alpha) \tilde{s}_{\text{cos}}$	C (0.718)
3	Reciprocal Rank Fusion	$\sum 1 / (k + \text{rank}_e)$	A, B (0.0323)
4	Peak vs consistency	convexity of $1 / (k + r)$	opposite picks

7 Equation cheat-sheet

Min-max normalise	$\tilde{s} = (s - s_{\min}) / (s_{\max} - s_{\min})$
Weighted-sum fusion	$\alpha \tilde{s}_{\text{lex}} + (1 - \alpha) \tilde{s}_{\text{sem}}$
Reciprocal Rank Fusion	$\sum_e 1 / (k + \text{rank}_e(d))$
RRF default constant	$k = 60$
Behaviour	weighted = average; RRF = peak

A Reference implementation

```
def minmax(scores):
    lo, hi = min(scores.values()), max(scores.values())
    return {k: (v - lo) / (hi - lo) for k, v in scores.items()}

def weighted(bm25, cos, a=0.5):
    nb, nc = minmax(bm25), minmax(cos)
    return {d: a*nb[d] + (1-a)*nc[d] for d in bm25}

def rrf(ranks_list, k=60):
    out = {}
    for ranks in ranks_list:
        for d, r in ranks.items():
            out[d] = out.get(d, 0) + 1/(k + r)
    return out

bm25 = {"A":1.94, "B":0.50, "C":1.20}
cos = {"A":0.20, "B":0.99, "C":0.95}
print(weighted(bm25, cos))          # C 0.718 tops A=B 0.500
print(rrf([{"A":1,"B":3,"C":2}, {"A":3,"B":1,"C":2}])) # A=B>C
```

— end of document —